

TP3 NoSQL : Tests de Sécurité

Validation des Permissions - Partie 3

Travail Pratique N°3 - Module NoSQL



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Test de l'utilisateur lecture seule (soc_analyst_ro)	3
2.1	Connexion avec l'utilisateur lecture seule	3
2.2	Tests effectués et résultats	3
2.3	Interprétation des résultats	3
3	Test de l'utilisateur manager (soc_manager)	4
3.1	Connexion avec l'utilisateur manager	4
3.2	Tests effectués et résultats	4
3.3	Analyse de la séparation des tâches	4
4	Test de l'utilisateur applicatif (soc_app)	4
4.1	Connexion avec l'utilisateur applicatif	4
4.2	Tests effectués et résultats	5
5	Résumé des tests de sécurité	5
5.1	Validation des objectifs de sécurité	5
6	Conclusion	6
	Captures d'écran	7

1 Introduction

Contexte - Tests de sécurité

Cette troisième et dernière partie du projet SOC-lite est consacrée aux **tests de sécurité** et à la validation des permissions configurées. Après avoir déployé MongoDB sécurisé, configuré l'authentification, créé des rôles personnalisés et des utilisateurs dédiés, nous devons maintenant vérifier que chaque utilisateur ne dispose que des droits qui lui ont été attribués.

Les tests suivants valident le principe du moindre privilège et garantissent que la séparation des tâches est correctement implémentée.

2 Test de l'utilisateur lecture seule (soc_analyst_ro)

L'utilisateur `soc_analyst_ro` dispose uniquement du rôle `read` sur la base `soc_lite`. Ce test vérifie que les permissions sont correctement appliquées.

2.1 Connexion avec l'utilisateur lecture seule

```
1 mongosh -u soc_analyst_ro -p AnalystR0@2026! --  
  authenticationDatabase soc_lite
```

2.2 Tests effectués et résultats

— Lecture autorisée :

```
1 use soc_lite  
2 db.security_logs.find().limit(5)
```

Résultat : La requête s'exécute correctement et retourne les logs.

— Écriture refusée :

```
1 db.incidents.insert({ title: "Test", severity: "high" })
```

Résultat : MongoDB retourne une erreur indiquant que l'utilisateur n'est pas autorisé à effectuer cette opération.

2.3 Interprétation des résultats

Ce test confirme que le rôle `read` est correctement implémenté. Un analyste peut consulter les incidents et logs existants pour faire son travail d'investigation, mais ne peut pas modifier ou supprimer des données. Cette limitation protège l'intégrité des données en cas d'erreur ou de compromission du compte analyste.

3 Test de l'utilisateur manager (soc_manager)

L'utilisateur `soc_manager` dispose du rôle personnalisé `incidentManager` qui permet les opérations CRUD uniquement sur la collection `incidents`.

3.1 Connexion avec l'utilisateur manager

```
1 mongosh -u soc_manager -p Mgr$ecure2026! --authenticationDatabase  
  soc_lite
```

3.2 Tests effectués et résultats

— Accès aux incidents autorisé :

```
1 use soc_lite  
2 db.incidents.find()  
3 db.incidents.insert({ title: "Nouvel incident", status: "  
  open" })  
4 db.incidents.updateOne({ title: "Nouvel incident" }, {  
  $set: { status: "resolved" } })  
5 db.incidents.remove({ title: "Nouvel incident" })
```

Résultat : Toutes les opérations CRUD sur `incidents` réussissent.

— Accès refusé aux autres collections :

```
1 db.security_logs.find()
```

Résultat : MongoDB refuse l'accès à la collection `security_logs` car le rôle `incidentManager` ne comprend pas d'actions sur cette ressource.

3.3 Analyse de la séparation des tâches

La restriction d'accès aux autres collections est fondamentale dans un SOC :

- Les **security_logs** contiennent des informations sensibles sur les alertes brutes
- Les **users** contiennent des données personnelles des analystes
- Le manager peut gérer les incidents sans avoir accès à ces données potentiellement sensibles

4 Test de l'utilisateur applicatif (soc_app)

L'utilisateur `soc_app` dispose du rôle `readWrite` sur toute la base `soc_lite`. Ce compte est utilisé par le backend Node.js.

4.1 Connexion avec l'utilisateur applicatif

```
1 mongosh -u soc_app -p AppP@ss2026! --authenticationDatabase  
  soc_lite
```

4.2 Tests effectués et résultats

- Lecture autorisée sur toutes les collections
- Écriture autorisée sur toutes les collections
- L'application fonctionne normalement

Le backend peut ainsi :

- Enregistrer les nouveaux logs de sécurité dans `security_logs`
- Créer, lire, modifier et supprimer des incidents dans `incidents`
- Gérer les utilisateurs dans `users`

5 Résumé des tests de sécurité

Utilisateur	Lecture	Écriture	Accès restreint à
soc_analyst_ro	Toutes collections		N/A
soc_manager	Uniquement incidents	Uniquement incidents	security_logs, users
soc_app	Toutes collections	Toutes collections	Aucune

5.1 Validation des objectifs de sécurité

- Authentification obligatoire pour accéder à MongoDB
- Séparation des privilèges entre les utilisateurs
- Principe du moindre privilège respecté
- RBAC correctement implémenté avec un rôle personnalisé

6 Conclusion

Bilan de la sécurisation MongoDB

La sécurisation de MongoDB dans ce projet SOC-lite permet de garantir la **confidentialité** et l'**intégrité** des données. L'utilisation du RBAC constitue une bonne pratique essentielle pour toute application manipulant des données sensibles.

Points clés réalisés :

- Déploiement Docker avec authentification activée
- Exposition limitée à localhost uniquement
- Création d'un rôle personnalisé **incidentManager**
- Trois utilisateurs avec des privilèges distincts
- Backend connecté avec un compte dédié (non admin)
- Tests de validation des permissions

Perspectives d'amélioration :

- **Chiffrement TLS** : Pour sécuriser les communications réseau entre l'application et MongoDB
- **Audit logs MongoDB** : Pour tracer toutes les opérations sensibles
- **Monitoring avancé** : Mise en place d'alertes sur les tentatives d'accès anormales
- **Chiffrement au repos** : Protection des données stockées sur disque

Fin du TP3 NoSQL

Captures d'écran

```

soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69d1b34f27551dd738edcd6a'),
    incident_id: 'INC-2025-001',
    title: 'Brute force SSH sur srv-web-01',
    description: 'Multiples tentatives SSH échouées depuis 192.168.1.105',
    severity: 'high',
    status: 'investigating',
    assigned_to: 'alice_analyst',
    tags: [ 'brute-force', 'ssh' ],
    related_log_ids: [
      ObjectId('69d1b34f27551dd738edcd64'),
      ObjectId('69d1b34f27551dd738edcd65')
    ],
    comments: [
      {
        author: 'alice_analyst',
        text: 'Pattern brute-force confirmé.',
        timestamp: ISODate('2025-03-03T10:35:00.000Z')
      }
    ],
    mitre_tactics: [ 'TA0006' ],
    mitre_techniques: [ 'T1110.001' ],
    created_at: ISODate('2026-04-05T00:56:47.988Z'),
    updated_at: ISODate('2026-04-05T00:56:47.988Z')
  },
  {
    _id: ObjectId('69d1b34f27551dd738edcd6b'),
    incident_id: 'INC-2025-002',
    title: 'Malware détecté sur endpoint-pc-042',
    description: 'Trojan détecté avec suspicion d'exfiltration',
    severity: 'critical',
    status: 'active',
    assigned_to: 'bob_senior',
  }
]

```

FIGURE 1 – Figure 1

```

soc_lite> docker exec -it mongo_secure mongosh -u soc_analyst_ro -p AnalystR0@2026! --authenticationDatabase s
soc_lite> db.security_logs.find().limit(2)

soc_lite> db.incidents.find({severity: "high"})
[
  {
    _id: ObjectId('69d1b34f27551dd738edcd6a'),
    incident_id: 'INC-2025-001',
    title: 'Brute force SSH sur srv-web-01',
    description: 'Multiples tentatives SSH échouées depuis 192.168.1.105',
    severity: 'high',
    status: 'investigating',
    assigned_to: 'alice_analyst',
    tags: [ 'brute-force', 'ssh' ],
    related_log_ids: [
      ObjectId('69d1b34f27551dd738edcd64'),
      ObjectId('69d1b34f27551dd738edcd65')
    ],
    comments: [
      {
        author: 'alice_analyst',
        text: 'Pattern brute-force confirmé.',
        timestamp: ISODate('2025-03-03T10:35:00.000Z')
      }
    ],
    mitre_tactics: [ 'TA0006' ],
    mitre_techniques: [ 'T1110.001' ],
    created_at: ISODate('2026-04-05T00:56:47.988Z'),
    updated_at: ISODate('2026-04-05T00:56:47.988Z')
  }
]

```

FIGURE 2 – Figure 2

```

PS D:\4eme-année\application SOC-lite> docker exec -it mongo_secure mongosh -u soc_app -p App@ss2026! --authenticationDatabase soc_lite
Current Mongosh Log ID: 69d2428dad234ba6678563b0
Connecting to: mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=soc_lite&appName=mongosh+2.7.0
Using MongoDB: 7.0.30
Using Mongosh: 2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertOne({ test:"ok"})
{
  acknowledged: true,
  insertedId: ObjectId('69d24385ad234ba6678563b1')
}
soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69d1b34f27551dd738edcd6a'),
    incident_id: 'INC-2025-001',
    title: 'Brute force SSH sur srv-web-01',
    description: 'Multiples tentatives SSH échouées depuis 192.168.1.105',
    severity: 'high',
    status: 'investigating',
    assigned_to: 'alice_analyst',
    tags: [ 'brute-force', 'ssh' ],
    related_log_ids: [
      ObjectId('69d1b34f27551dd738edcd64'),
      ObjectId('69d1b34f27551dd738edcd65')
    ]
  }
]

```

FIGURE 3 – Figure 3

```

soc_lite> db.incidents.updateOne(
|   { incident_id: "INC-2025-001" },
|   { $set: { status: "resolved" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

FIGURE 4 – Figure 4